DynamicServiceOrchestration

System Description

Abstract

This document describes the DynamicServiceOrchestration core system of the Eclipse Arrowhead framework (documentation edition 5.3.0), as implemented in Java Spring in `ah5-core-java-spring/serviceorchestration-dynamic`, including the local extensions of this workspace.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Significant Prior Art 4
2.2 How This System Is Meant to Be Used 4
2.3 System functionalities and properties 4
2.4 Important Delimitations 5

3 Services **6**
3.1 Produced services 6
3.2 Consumed services 6

4 Security **8**
4.1 Security model 8

5 References **8**

6 Revision History **10**
6.1 Amendments 10
6.2 Quality Assurance 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the DynamicServiceOrchestration system. DynamicServiceOrchestration finds, at request time, the providers a consumer shall use. It discovers candidates in the ServiceRegistry, keeps only the authorized, not blacklisted and not locked ones with matching interfaces, applies preferences, QoS requirements, matchmaking and exclusivity, obtains the authorization tokens the providers require, and returns the result; it also pushes results to subscribers. In this workspace it extends orchestration to neighbour clouds through the Gatekeeper, and enforces the active-plant consumer/service scopes submitted by PlantDescriptionEngine.



Figure 1: SysML-style block view of DynamicServiceOrchestration with its produced and consumed services.

The rest of this document is organized as follows. In Section 2.1, we reference major prior art capabilities of the system. In Section 2.2, we describe the intended usage of the system. In Section 2.3, we describe fundamental properties provided by the system. In Section 2.4, we describe delimitations of capabilities of the system. In Section 3, we describe the abstract service functions consumed or produced by the system. In Section 4, we describe the security capabilities of the system.

2.1 Significant Prior Art

Successor of the dynamic orchestration of the Arrowhead 4.x Orchestrator. Arrowhead 5 adds operation-level requirements, metadata and interface property requirements, exclusivity with locks, QoS through DeviceQoSEvaluator, interface translation through TranslationManager, and push orchestration with subscriptions.

2.2 How This System Is Meant to Be Used

A consumer calls `serviceOrchestration.pull` with the service definition and, optionally, operations, versions, interface templates, metadata requirements, preferred providers, flags, QoS requirements and an exclusivity duration. It uses the returned interface (and token) to call the provider. Support systems manage push subscriptions, locks and history with the management services. All Java systems of this workspace reach non-bootstrap services by pulling from DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`).

2.3 System functionalities and properties

2.3.1 Functional properties of the system

- Pull orchestration with the flags `MATCHMAKING`, `ONLY_PREFERRED`, `ONLY_EXCLUSIVE`, `ALLOW_TRANSLATION`, `ALLOW_INTERCLOUD`, `ONLY_INTERCLOUD`.
- Authorization cross-check (`authorizationManagement.check-policies`) and token generation (`authorizationTokenManagement.generate-tokens`).
- Blacklist cross-check, QoS evaluation (DeviceQoSEvaluator), interface translation (TranslationManager), each optional.
- Exclusive reservation of providers with orchestration locks.
- Push orchestration: subscriptions, triggers, worker threads sending results to HTTP(S) or MQTT(S) notify interfaces.
- Orchestration history of every pull and push job; in this workspace with the selected providers in the job message.
- Inter-cloud orchestration through the local Gatekeeper (`globalServiceDiscovery.query`, `interCloudNegotiation.negotiate`; local extension).

2.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8441	HTTPS port.
`authentication.policy`	declared	Test cloud: `outsourced`.
`arrowhead.security.profile`	secure	`development` selects the isolated development exception; never set in a deployment.
`management.policy` / `management.whitelist`	authorization	Access to the management services.
`enable.authorization`	true	Authorization cross-check and token generation.
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist filter and cross-check (test cloud: true).
`enable.qos`	false	QoS evaluation with DeviceQoSEvaluator.

Property	Default	Meaning
`enable.translation`	false	Interface translation with TranslationManager.
`enable.intercloud`	false	Inter-cloud orchestration through the Gatekeeper (test cloud: true).
`http.collector.bootstrap.services`	SR services, identity, serviceOrchestration, authorization*, blacklist*	Services looked up in the ServiceRegistry instead of orchestrated (local).
`orchestration.history.max.age`	15	Days the history is kept.
`cleaner.job.interval`	30000	Cleaner interval in ms (subscriptions, history, locks).
`push.orchestration.max.thread`	5	Push worker threads.
`normalization.mode`	extended	Name normalization.
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`	false	Publish native sysMonitor observations (test cloud: true).
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (local).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; in the test clouds they are per-cloud secrets mounted as files.

2.3.3 Data stored by the system

An atomic active-plant rule snapshot (orchestration_plant_rules), orchestration jobs (history), push subscriptions, orchestration locks, QoS evaluation results, and log entries (database `ah_serviceorchestration_dynamic`).

2.3.4 Non functional properties

A pull makes several synchronous calls (ServiceRegistry lookup, authorization check, optionally Blacklist, Device QoSEvaluator, TranslationManager, Gatekeeper, token generation); its latency is the sum of these. Push orchestration runs in a bounded thread pool.

2.3.5 Stateful or stateless

Stateful: history, subscriptions and locks are persistent; an orchestration itself is computed from the current state of the other systems.

2.4 Important Delimitations

DynamicServiceOrchestration selects providers but does not mediate the consumption, except through a translation bridge or an inter-cloud GateTunnel session. It cannot orchestrate the services its own orchestration depends on (ServiceRegistry, ConsumerAuthorization, Blacklist); those are bootstrap services. A missing authorization grant silently yields no result. Managed plant scopes constrain the provider instance set before normal checks; lower priority selects the best surviving provider. Empty scopes and ONLY_INTERCLOUD cannot escape to another cloud. This local extension needs database_patch_pde_dso.sql before startup on an existing database.

3 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

3.1 Produced services

Service	Version	Interfaces	SD	IDD
`serviceOrchestration`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`serviceOrchestrationPush Management`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`serviceOrchestrationLock Management`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`serviceOrchestrationHistory Management`	1.0.0	HTTP(S), MQTT(S)	[10]	[11], [12]
`serviceOrchestrationPlant Management`	1.0.0	HTTP(S)	[13]	[14]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[15]	[16], [17]
`sysMonitor`	1.0.0	AMQP(S)	[18]	[19]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. Upstream 5.2.2 also answers `GET <base>/monitor/echo` on some systems; this endpoint is not registered as a service.

3.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	serviceDiscovery	register, revoke, lookup	Registration; discovery of candidates (direct access).	always
Authentication	identity	identity-login, identity-verify	Own token; verification of requester tokens.	`authentication. policy= outsourced`
Consumer Authorization	authorization; authorization Management	verify; check-policies	Management filter; authorization cross-check of candidates.	`enable. authorization= true`
Consumer Authorization	authorizationToken Management	generate-tokens	Tokens for provider interfaces with token policies.	`enable. authorization= true`
Blacklist	blacklistDiscovery; blacklistManagement	check; query	Blacklist filter; cross-check of candidates.	`enable. blacklist. filter=true`
DeviceQo SEvaluator	qualityEvaluation	filter, sort	QoS requirements.	`enable.qos=true`
TranslationManager	translationBridge Management	discovery, negotiation	Interface translation.	`enable. translation= true`

Provider	Service	Operations	Use	Condition
Gatekeeper	globalService Discovery; interCloud Negotiation	query; negotiate	Inter-cloud orchestration (local extension).	`enable. intercloud=true`
Subscribers	notify interface	HTTP(S) request or MQTT(S) publish	Push orchestration results.	subscriptions
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor. enabled=true`

Except for the bootstrap services (`http.collector.bootstrap.services`), which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for `http.collector.orchestration.ttl` = 300 s); every such call needs a ConsumerAuthorization grant.

4 Security

The default is the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The Arrowhead-test-cloud-java deployment runs the secure mode.

The packaged `application.properties` is a development sample that disables TLS and declares identities. The system refuses to start with it, or with any other effective configuration that sets `server.ssl.enabled=false`, `authentication.policy=declared`, `service.authorization.enabled=false` or, while monitoring is on, `sysmonitor.ssl.enabled=false`. The check runs before the system accepts a connection. Only an explicit `arrowhead.security.profile=development` lets such a system start, for an isolated test, and it then logs a non-conformance warning.

4.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates (`server.ssl.client-auth=need`); MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the test cloud. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate` (client certificate CN), `outsourced` (identity token verified with Authentication), `internal` (Authentication only) or `declared` (insecure). In this workspace an identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL when `arrowhead.ssl.crl-file` is set (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION` via ConsumerAuthorization); an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- The orchestrator only returns providers the consumer is authorized for, and generates the tokens of token-protected interfaces on the consumer's behalf (it is in `unbounded.token.generation.whitelist` of ConsumerAuthorization). Its access to the ServiceRegistry relies on `service.discovery.direct.access` and the management whitelist.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

5 References

[1] J. Delsing, "serviceOrchestration – Service Description (DynamicServiceOrchestration)," Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-SD.pdf.

[2] ——, "serviceOrchestration HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration)," Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-HTTP-IDD.pdf.

[3] ——, "serviceOrchestration MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration)," Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-MQTT-IDD.pdf.

[4] ——, "serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration)," Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-SD.pdf.

[5] ——, "serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration)," Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

[6] ——, “serviceOrchestrationPushManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-MQTT-IDD.pdf.

[7] ——, “serviceOrchestrationLockManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-SD.pdf.

[8] ——, “serviceOrchestrationLockManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-HTTP-IDD.pdf.

[9] ——, “serviceOrchestrationLockManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-MQTT-IDD.pdf.

[10] ——, “serviceOrchestrationHistoryManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-SD.pdf.

[11] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

[12] ——, “serviceOrchestrationHistoryManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-MQTT-IDD.pdf.

[13] ——, “serviceOrchestrationPlantManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPlantManagement-SD.pdf.

[14] ——, “serviceOrchestrationPlantManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPlantManagement-HTTP-IDD.pdf.

[15] ——, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-SD.pdf.

[16] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-MQTT-IDD.pdf.

[18] ——, “sysMonitor – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-sysMonitor-SD.pdf.

[19] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-sysMonitor-AMQP-IDD.pdf.

6 Revision History

6.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

6.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	